pillar-UDP: The Portable Cell-Minted Session Key — Transport-Frame Encryption without a Handshake

pillar engineering (machine-checked design note)

2026-09-09

Abstract

pillar-UDP is a bare datagram substrate with no built-in cryptography, whose posture is *redundant datagrams sprayed across multiple paths, deduplicated by content address, processed once*, served by ingress nodes that fail over and load-balance (companion paper on congestion). A connection-oriented transport handshake—libp2p Noise, or a per-pair WireGuard-style session—fights that posture: its ordered multi-message exchange needs a reliability sublayer just to land over lossy, unordered UDP, and it binds a session to *one pair* of endpoints, so a redundant copy that arrives at a different node, or an ingress failover, cannot be served without re-handshaking. We instead treat the *cell* as a key-distribution centre. A client picks a fresh ephemeral X25519 key; the transport session key $K_s = \mathrm{HKDF}(\mathrm{ECDH}(eph, cell\_static), nonce)$ is *deterministically* derived, so the client and *every* cell node compute the byte-identical K_s with no round-trip and no race. The authorization is a *cell-signed record on streamdb* that converges cell-wide; K_s is never stored, only its derivation inputs, and any node recomputes it—making the session *portable* across every node (ingress failover, relay, load-balancing). Sender identity comes from the ed25519 signature inside the `PillarMessage`, not from K_s (which only proves cell membership); replay defence is the content-addressed dedup pillar already runs; forward secrecy is deliberately bounded to cell-key security plus a *security-critical* erase-on-revoke. Anonymity is a *policy* state, not a separate crypto path: an anonymous key runs the identical flow and is confined by default-deny WoT/RBAC. The design is machine-checked in TLA$^+$ (`PillarUdpEncryption.tla`, green under TLC before any Rust): the session converges on one cell-signed key, a node serves only an authorized session, replay is idempotent, an unattested principal never gains a privileged grant, and a revoked key is eventually erased. This scheme applies *only* to pillar-UDP; QUIC and TCP fall back to their own TLS 1.3.

1 Why not Noise, and why not a per-pair session

Two candidate transport-crypto designs were rejected against pillar-UDP's multipath, any-ingress posture.

libp2p Noise upgrade is a *session/connection* protocol. Its handshake is a strict ordered multi-message exchange that needs a reliability sublayer just to land over lossy, unordered UDP, and it binds a session to *one pair* of endpoints. That fights the connectionless, multipath, any-ingress model.

A per-pair Noise/WireGuard session gives forward secrecy and replay protection, but a session bound to a pair is *not portable*: if the first ingress node dies, or a redundant copy arrives at a different node, the session cannot be served without re-handshaking.

The blocker to portability is precise and cryptographic. An AEAD session is *a shared key plus a per-direction monotonic nonce counter*. The *key* is portable; the *counter* is not—two nodes incrementing one counter would reuse a nonce, and AEAD nonce reuse is a total break (keystream reuse ⇒ plaintext recovery; authentication-key leak ⇒ forgery). Remove the shared counter and derive the nonce collision-free instead—convergent (`HKDF(`K_s`, content_address(frame))`) or sender-partitioned (`node_id` ‖ `counter`)—and the key becomes portable across every node that holds it. That is the whole idea.

2 The scheme: the cell is a KDC, the session key rides streamdb

pillar already has a shared trust boundary (the *cell*, a genesis principal with a static key), a converging cell-internal state substrate (*streamdb*), per-message *ed25519 signatures*, and *content-addressed dedup*—exactly the ingredients a portable session needs. So the transport session key is built from them: it is portable and cell-scoped (every node can derive and serve it, no per-pair binding); it is *deterministically derived*, not minted-and-reconciled (two ingress nodes handling the same sprayed init derive the byte-identical key—no race); it is *authorized by a cell-signed streamdb record* that converges to every node (so failover / relay / load-balancing all serve one session); and anonymity is a policy state, not a separate scheme (§5). The end-to-end exchange is drawn in Fig. 1 and as a time sequence in Fig. 2.

2.1 Key derivation (deterministic, portable)

A client—a node, a non-node client, or an anonymous client—opens a session by choosing a fresh ephemeral X25519 keypair (eph_pk, eph_sk) and a random `client_nonce`. The session key is a static-ephemeral

ECDH against the cell static key, run through an HKDF with a domain separator distinct from the content seal:

$$\begin{aligned} \mathit{shared} &= \mathrm{X25519}(\mathit{eph\_sk},\ \mathit{cell\_static\_pk}) \\ &= \mathrm{X25519}(\mathit{cell\_static\_sk},\ \mathit{eph\_pk}) \\ K_s &= \mathrm{HKDF}(\mathit{shared},\ \mathrm{salt}{=}\mathit{client\_nonce}, \\ &\qquad \mathrm{info}{=}\text{“pillar-udp/session/v1”}) \\ \mathit{sid} &= \mathrm{addr}(\mathit{eph\_pk} \parallel \mathit{client\_nonce}) \end{aligned}$$

The first line is the client's derivation, the second any cell node's—equal by the Diffie–Hellman symmetry, so no key crosses the wire. Both endpoints compute the *same* K_s with no key on the wire and no round-trip: the client derives it the instant it picks *eph_pk* (it knows the published *cell_static_pk*); *every* cell node derives the identical K_s from (*cell_static_sk*, *eph_pk*, *client_nonce*) carried in the session record. Because K_s is a pure function of those inputs, two racing ingress nodes cannot diverge—the "collision" is byte-identical, so there is nothing for the client to reject. K_s is *never at rest*: streamdb stores only the derivation inputs plus the cell authorization, and any node recomputes K_s on demand. Per-datagram AEAD nonces under K_s are collision-free without coordination (convergent or sender-partitioned), so the many cell nodes sharing K_s never reuse a nonce.

2.2 The session record (cell-signed, on streamdb)

The record carries no key material—only the inputs from which K_s is derivable by a cell-secret holder, plus the cell signature that any node verifies before honoring the session:

```
SessionRecord {
  session_id  :  Cid // addr(eph_pk‖nonce)
  principal_pk :  PublicKey // node/user/anon
  eph_pk      :  X25519PublicKey
  client_nonce :  bytes
  grants      :  PolicySet // WoT/RBAC §5
  not_after   :  Timestamp
  cell_sig    :  Signature // signed by CELL key
}
```

streamdb convergence makes the record available cell-wide.

3 What each layer provides (and deliberately does not)

Confidentiality of framing—K_s-AEAD on every pillar-UDP datagram. (Application content is *additionally* cell-sealed end-to-end, `pillar-message-format` §5.) **Sender identity**—*not* from K_s (which only proves cell membership, since every node holds it) but from the ed25519 signature inside the `PillarMessage`; the transport key gives group-level confidentiality, identity is a content-layer property. **Replay protection**—from content-addressed dedup: a replayed datagram has the same Cid and is dropped by the store pillar already runs; no per-session replay window is needed. **Portability / failover / load-balancing**—any node with the converged record derives the same K_s and serves the session. **Forward secrecy**—bounded to cell-key security plus erase-on-revoke, *by design*. K_s is derivable under the cell static key, so a cell-key compromise exposes past sessions; this is the identical tradeoff the content cell-seal already accepts. FS *independent* of the cell key would require a discarded ephemeral DH secret, which directly contradicts a portable, cell-derivable key—so it is deliberately traded for cross-node portability. Mitigations: periodic cell rekey, and erase on revoke.

4 Revocation and GC are security-critical

On close or timeout a cell node writes a cell-signed *revocation* for the `session_id` to streamdb; convergence stops every node from honoring K_s. GC of the revoked record is a *security operation, not hygiene*: a revoked-but-uncollected record keeps K_s derivable, i.e. a live decryption oracle for any captured ciphertext, so GC must erase within a bounded deadline. Ciphertext captured *before* revocation stays readable by anyone who held K_s—inherent to shared-key transport and accepted.

5 Anonymous sessions = the same scheme + policy

An anonymous client generates an anonymous signing key and runs the *exact scheme* of §2 and Fig. 1. The anonymous key is *cryptographically equivalent* to a user or node key and is subject to the *same WoT/RBAC checks* as any principal. The only difference is the *policy outcome*: absent signed role attestations for the key, the cell's default-deny RBAC withholds sensitive data and privileged operations—the anonymous session can communicate, but only within an unattested principal's grant set. Nothing else about key derivation, portability, revocation, or nonce discipline changes. Two consequences: *anonymous ⇒ pseudonymous, not automatically unlinkable*—a stable anonymous key lets the cell link a client across sessions, so unlinkability is a client choice (fresh key, hence fresh `session_id`, per session); and *the policy gate is the whole security boundary for anonymity*—it rests on the ROI invariant that every controller enforces WoT/RBAC default-deny, so an unattested principal must never acquire a privileged grant.

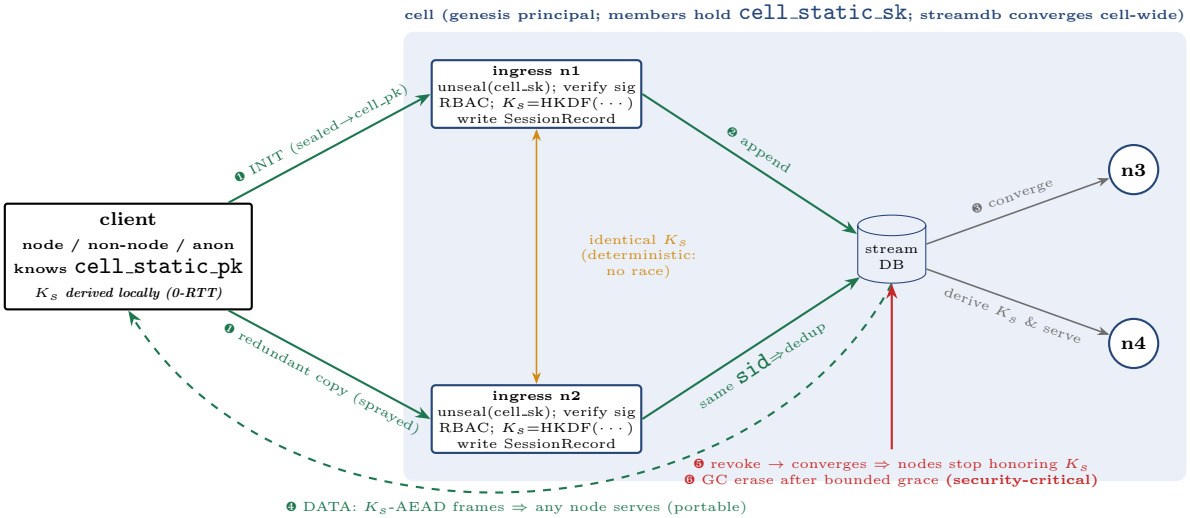

Figure 1: Packet flow. ❶ The client sprays a cell-sealed init (fresh ephemeral + nonce) over redundant paths; either or both ingress nodes process it and, because K_s is a deterministic function of (*cell_sk*, *eph_pk*, *nonce*), derive the *byte-identical* key. ❷ Each writes a cell-signed `SessionRecord`; identical `session_id` means the store dedups to one record. ❸ streamdb converges the record cell-wide, so nodes `n3`, `n4` can now derive K_s and serve the session. ❹ Data frames are K_s-AEAD-sealed with a convergent nonce and carry an inner `PillarMessage` signed by the client's key (sender identity); any node holding the record serves them, and a replayed frame has the same Cid and is deduped. ❺ On close/timeout a cell-signed revocation converges and every node stops honoring K_s. ❻ GC erasing the record is a *security* operation, not hygiene. A non-node client and an anonymous client run the identical flow; only the RBAC outcome differs (§5). Modeled in `PillarUdpEncryption.tla`.

6 Compatibility & rollout

Dropping the libp2p Noise upgrade is a *breaking* pillar-UDP change, gated by the versioning spine: the pillar-UDP protocol version bumps and compatibility negotiation refuses a legacy-Noise peer cleanly rather than misframing. A mixed legacy-Noise / session-key swarm must coexist through the rollout window (`RollingCoexistence`, `NegotiationRefusesIncompatible` in the message-format spec). QUIC/TCP peers are unaffected—they never used this path.

7 Machine-checked obligations

The design is TLA$^+$-gated (method #1): `PillarUdpEncryption.tla` must be green under TLC before any Rust. Table 1 lists the obligations; all pass, together with the full specs gate and the schema round-trip.

8 Relationship to the content seal

Independent layers, composed. The *content seal* (`pillar-message-format` §5) is end-to-end to the cell, convergent, transport-agnostic, and gives the stable Cid. The *transport frame seal* (this paper) is pillar-UDP-specific, keyed by the portable session key, and protects on-wire framing. A pillar-UDP datagram carries a cell-sealed body inside a session-key-sealed frame; on QUIC/TCP the outer frame seal is the transport's TLS instead, and the content seal is unchanged.

References

[1] T. Perrin, *The Noise Protocol Framework*, Rev. 34, 2018.

[2] J. A. Donenfeld, *WireGuard: Next Generation Kernel Network Tunnel*, NDSS, 2017.

[3] H. Krawczyk, *Cryptographic Extraction and Key Derivation: The HKDF Scheme*, CRYPTO, 2010.

[4] L. Lamport, *Specifying Systems*, Addison-Wesley, 2002.

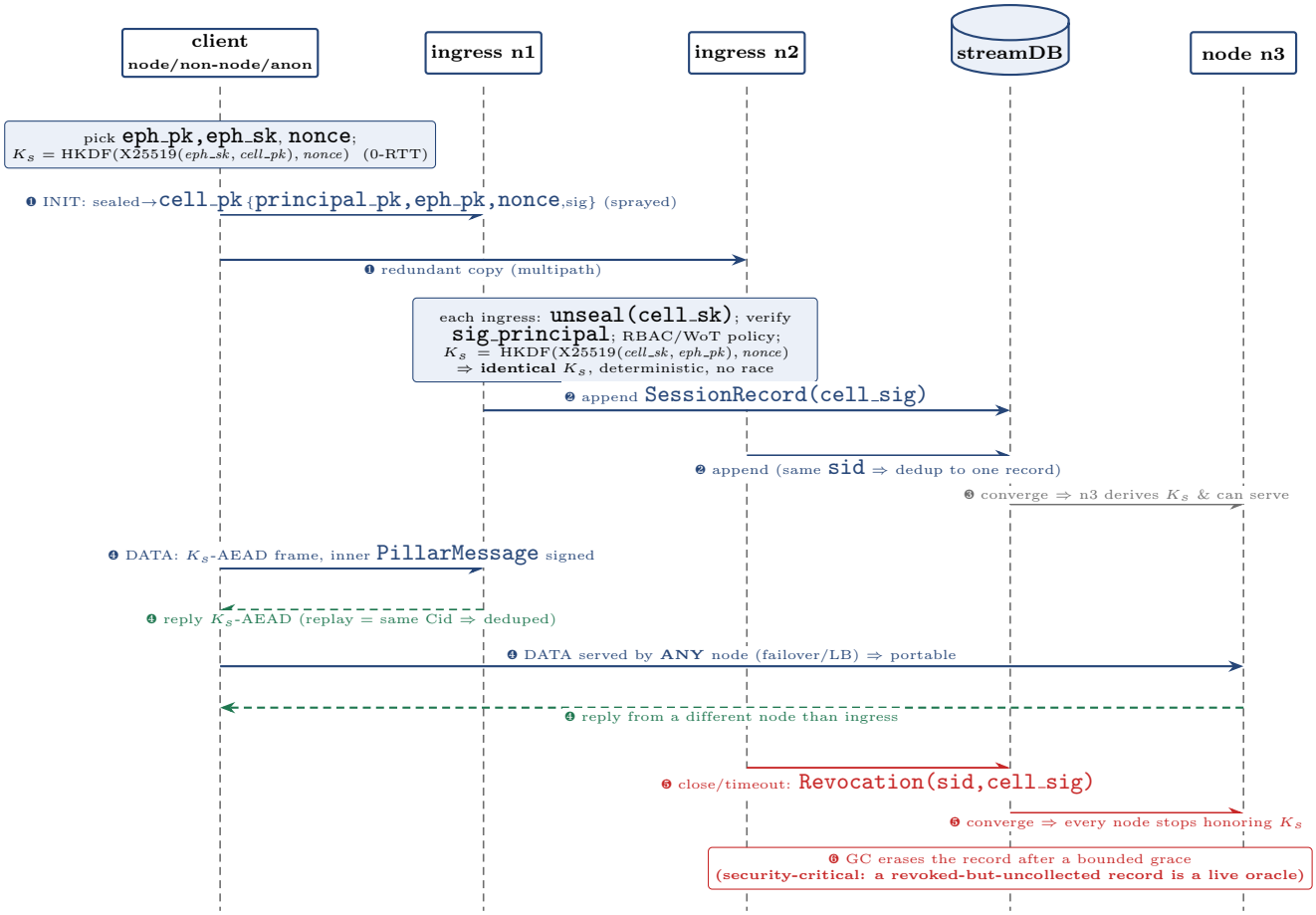

Figure 2: Time sequence. The same flow as Fig. 1, ordered in time. The client derives K_s locally the instant it picks its ephemeral (0-RTT), then sprays the cell-sealed init over redundant paths. *Both* ingress nodes can process the init and, running the mirror-image ECDH against the cell static key, derive the *byte-identical* K_s—so the two `SessionRecord` appends carry the same `sid` and streamdb dedups them to one cell-signed record. Convergence lets a node (`n3`) that never saw the init derive K_s and serve the session, which is why data frames are portable: a reply may come from a *different* node than ingested the client. On close or timeout a cell-signed revocation converges and every node stops honoring K_s; GC then erasing the record is a security operation, since a revoked-but-uncollected record keeps K_s derivable. An anonymous client follows the identical sequence—only the RBAC policy outcome at each ingress differs (§5).

Invariant	Guarantee
`SessionKeyCellSigned`	every honored session is backed by a cell-signed record
`SessionConvergesOnOneKey`	$\leq$ 1 active authorization per `session_id`; every node derives the identical K_s the client did
`SessionAuthorizedBeforeServe`	a node serves only a session it has actually converged (no fabricated/unauthorized sessions)
`AnonIsUnattestedPrincipal`	a principal with no attestations holds only the restricted grant; never a privileged one
`SharedKeyReplayViaDedup`	applying a frame is content-addressed & idempotent; a replay causes no second effect
`RevokedKeyEventuallyErased` *(liveness)*	a revoked session's record is eventually erased from streamdb and every node's view
`NonceCollisionFree`	the per-frame nonce under K_s is injective in the plaintext (AEAD safety); discharged as a constant assumption

Table 1: TLA$^+$ obligations of `PillarUdpEncryption.tla`. Forward secrecy is a *documented, accepted* property (§4), not a state invariant: it is a statement about a hypothetical cell-key compromise, deliberately bounded rather than proven absent.